Introduction to Data Science
R exercises in-class
Week 1

Dr. Gah-Yi Ban
Imperial College Business School

Getting Started

NOTE This is an *R Notebook*, which combines text and code into a single document. The 14 lines are the settings of this R Notebook document- please do not change anything there. The text in a Notebook can be given special formatting using something called *Markdown*; it is not important to understand right away - we will take a closer look later in the module.

Before starting any coding practice, please follow the slides to create a project and copy the file handout01.Rmd into the project directory. Then open the file handout01.Rmd inside the project using the *bottom-right* "Files" panel.

Code in an R Notebook is kept in *code chunks* that appear in grey shaded boxes, like the one below.

```
2 + 3
```

```
## [1] 5
```

```
1 + 1
```

```
## [1] 2
```

#"'{trial}

#"'

Three backticks `symbols are used to open and close the code chunk. The parentheses after` such as {r Intro to Code Chunks} gives it a name; think of this as the tag or ID of the code chunk. Note that each code chunk should have a unique name.

Run a code chunk by placing your cursor on a line of code inside it, and clicking the **green arrow** on the top line of the code chunk. If you want to run one or more lines in a code chunk, select the line(s) you want to run, and use **Run** with a down arrow and selecting "Run Selected Line(s)".

If everything is working as expected, you should see the number 5 displayed above, in a *white area directly under the code chunk*.

Notice the [1] that precedes the result of the operation. For now, it is enough to know that this number in square brackets indicates output - we'll talk more about it when we start working with vectors.

Arithmetic

In the above code chunk, we used R as a calculator with 2 integer numbers. Use the code chunk below to try out the other common mathematical operators - * / with some float numbers.

```
# Addition with
3.5 + 2.4
```

```
## [1] 5.9
```

```
# Subtraction
3.5 - 2.4
```

```
## [1] 1.1
```

```
# Multiplication
3.5 * 2.4
```

```
## [1] 8.4
```

```
# Division
3.5 / 0.5
```

```
## [1] 7
```

In the code chunk above we introduced *comments*. A comment in R starts with a hash `#` symbol and is ignored during execution. Comments can exist on their own line, or can come immediately after some code.

Exercise 1: Order of Operations

R will follow the usual operator precedence that applies in mathematics. As expected, we can also apply BODMAS And BIDMAS in the code. For example, parentheses (round brackets) to force a specific order of evaluation.

In the code chunk below, *add* or *remove* parentheses so that all three expressions evaluate to the same result: 20

```
# Fix the parentheses below
(2 + 3 + 5)*2
```

```
## [1] 20
```

```
4*(10 - 5)
```

```
## [1] 20
```

```
2*8 + 4
```

```
## [1] 20
```

NOTE: Use **whitespace** above to improve the readability our code for everyone (both your future selves and your tutors!)

Best practice:

- a space between numbers and operators
- a space between operators and parentheses
- no space between numbers and parentheses

Variables

Often, it is useful to store data, or the result of an operation. Like other programming languages, R supports the *assignment* of *variables* for this.

For now, you can think of variables as having a name, and containing some data. To assign data to a variable in R, we use the *arrow assignment operator*.

```
# Assign a numeric value to x
x <- 3.14

# Assign the result of an operation to y
y <- 5 + 1.86

# Sum the variables and assign the result to z
z <- x + y

# Print output
print(z)
```

```
## [1] 10
```

NOTE: assigning data to variables does not produce any output. In the top- right pane of RStudio, the *Environment* window (**top-right** panel) will show any variables you created, so take a look there after running the above code chunk. Alternatively, we can always print a variable to see its value.

Now using the idea of variables, *add* or *remove* parentheses and mathematical operators in the code chunk below so that all three expressions evaluate to the same result: 20

```
# define variables
a <- 2
b <- 3
c <- 5
# Apply BIDMAS below to obtain 20 in each line
(a + b + c)*a
```

```
## [1] 20
```

```
(a + a)*(c*a - c)
```

```
## [1] 20
```

We can re-assign variables, but note that this will overwrite the original value.

```
# Assign a new numeric value to z
z <- 22

# Print z
print(z)
```

```
## [1] 22
```

```r
# Assign a new character value to z
z <- "Hello!"

# Print z
print(z)
```

```
## [1] "Hello!"
```

NOTE: Variables hold different *types* of data. In the above example, `z` variable was used to first hold numeric data, then character data.

The End